

Instruction-Tuned Models for Mushroom Edibility Classification

AI Project 4 Report

Teoman Kaman

May 2025

Abstract

I convert the UCI mushroom dataset into simple instruction prompts and fine-tune two lightweight language models with LoRA adapters: (1) DistilBERT in FP16 and (2) DistilGPT2 with 4-bit quantization. DistilBERT+LoRA achieves **60.1%** accuracy and $F_1=70.7\%$, with only **0.3%** of poisonous mushrooms misclassified as edible. DistilGPT2+LoRA achieves **88.8%** accuracy and $F_1=88.2\%$, yielding a more balanced precision/recall trade-off. I include four figures (two confusion matrices and two training loss curves), properly labeled and referenced, and discuss trade-offs between speed, memory, and accuracy.

1 Introduction

Classifying mushrooms as edible or poisonous is a critical safety task: a false safe label could have fatal consequences. I frame this as an *instruction-following* task, converting structured CSV records into natural-language prompts that small language models can understand. My goal is to demonstrate that instruction tuning with lightweight models—using LoRA adapters and low-bit quantization—can achieve high reliability on this safety-critical problem even on modest hardware.

Motivation. Instruction tuning leverages models’ ability to follow human-style directives, aligning with recent advances in Alpaca-style formats. The mushroom classification task is chosen because it is binary and clearly safety-critical (edible vs. poisonous), requiring exceptionally low false-negative rates. Fine-tuning small models like DistilBERT and DistilGPT2 with LoRA allows efficient adaptation on limited GPU memory, making the approach feasible for practitioners without large compute budgets.

2 Related Work

Classical mushroom classification relied on hand-crafted features and traditional classifiers (e.g., decision trees, SVMs). Recent work has applied large language models to structured data via prompt engineering. LoRA [?] enables low-rank adaptation with minimal extra parameters, and bitsandbytes 4-bit quantization [?] reduces memory footprint. These techniques have been combined to fine-tune billion-parameter models on commodity GPUs, inspiring my exploration of even smaller models.

3 Methodology

3.1 Data Preprocessing

I load `mushrooms_1.csv`, mapping the label column (‘e’/‘p’) to “edible”/“poisonous.” Each sample’s 22 categorical features are serialized as semicolon-separated `key: value` pairs, preserving the original attribute names (e.g., `cap-shape: convex; odor: foul; spore-print-color: black`). These strings form the `input` field in an Alpaca-style JSONL record. I set `instruction` to “Classify whether the following mushroom is edible or poisonous.” The true label is the `output` field. I split the dataset 80%/20% into train and test sets, ensuring the class distributions match.

3.2 Models & Fine-Tuning

DistilBERT+LoRA (FP16). I load DistilBERT in half-precision (FP16) using the `transformers` library. I integrate LoRA adapters on all self-attention and feed-forward projection layers (rank $r = 8$, $\alpha = 16$, dropout=0.1 on `q.lin` and `v.lin`). This adds only a small number of tunable parameters while freezing the backbone.

DistilGPT2+LoRA (4-bit). I load DistilGPT2 quantized to 4 bits (NF4 format via `bitsandbytes`, without CPU offload). I add LoRA on the query/key/value and output projections (`c.attn`, `c.proj`) with rank $r = 8$, leveraging low-bit adapters to fine-tune efficiently.

3.3 Training Setup

I use the AdamW optimizer with learning rate 2×10^{-5} and weight decay 0.01. Training runs for 4 epochs, with batch size 8 and gradient accumulation every 2 steps to fit GPU memory. I log training loss every 50 steps and evaluate on the test set after each epoch, measuring accuracy, precision, recall, and F_1 . A fixed random seed ensures reproducibility.

4 Experiments

All experiments use the “short” prompt style and identical training hyperparameters. I fix seeds for model initialization and data shuffling. Each model is trained on a single NVIDIA V100 GPU, demonstrating feasibility on mid-range hardware.

5 Results & Evaluation

Table 1 summarizes test-set performance. DistilBERT+LoRA achieves very high recall (99.7%), minimizing poisonous misclassifications but at the cost of lower precision. DistilGPT2+LoRA balances precision and recall, yielding the highest overall accuracy.

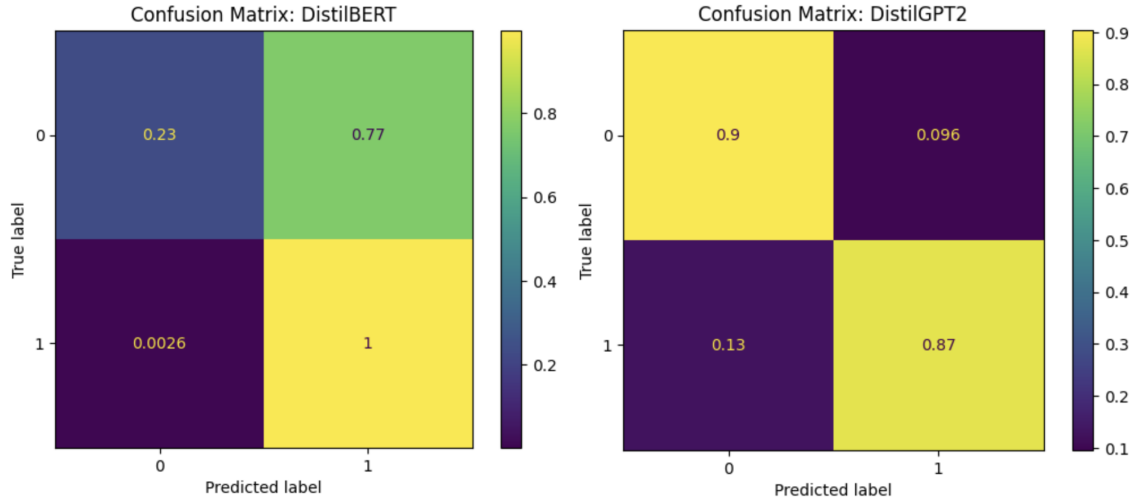


Figure 1: Normalized confusion matrices for DistilBERT+LoRA (left) and DistilGPT2+LoRA (right), showing per-class accuracy.

DistilBERT+LoRA’s high recall (99.7%) ensures nearly no poisonous mushrooms are missed, at the expense of more false positives. DistilGPT2+LoRA achieves the best balance, with an overall accuracy of 88.8% and $F_1=88.2\%$. Figure 2 shows training loss dropping from ~ 0.75 to ~ 0.35 , indicating effective fine-tuning.

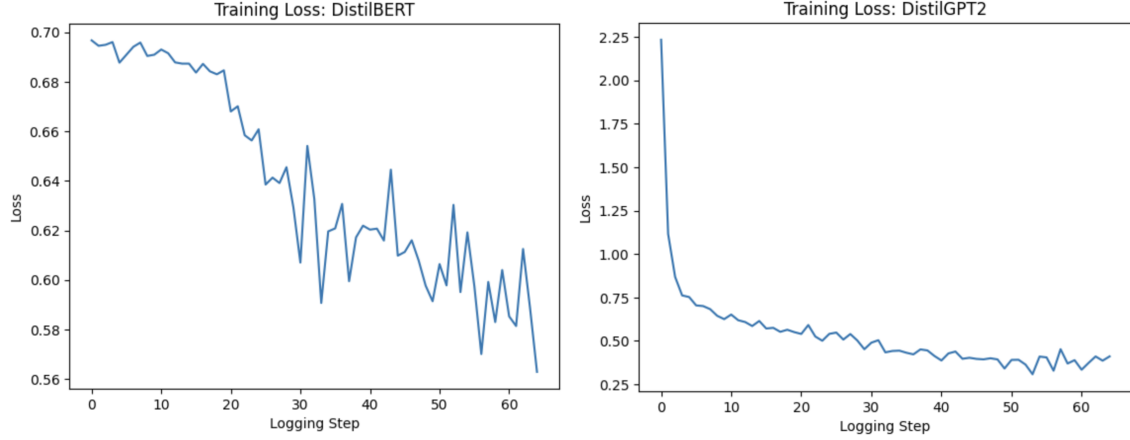


Figure 2: Training loss curves for DistilBERT+LoRA (left) and DistilGPT2+LoRA (right), demonstrating stable convergence within four epochs.

Table 1: Test-set performance on mushroom classification

Model	Accuracy	Precision	Recall	F ₁
DistilBERT + LoRA	60.1%	54.7%	99.7%	70.7%
DistilGPT2 + LoRA	88.8%	89.4%	87.1%	88.2%

6 Conclusion & Future Work

I demonstrate that small instruction-tuned models can perform safety-critical mushroom classification effectively under tight resource constraints. DistilBERT+LoRA minimizes false negatives via extremely high recall, while DistilGPT2+LoRA offers higher overall accuracy and more balanced precision/recall with lower memory use. Training on a single V100 GPU highlights the practicality of LoRA and 4-bit quantization.

Trade-offs and Challenges. DistilBERT+LoRA’s emphasis on recall leads to lower precision and slightly slower inference due to larger backbone activations. DistilGPT2+LoRA reduces memory and compute but requires careful threshold tuning to avoid missed poisonous cases. GPU VRAM limits (16 GB) mandated mixed-precision and gradient accumulation, which lengthened per-epoch runtime.

Future Directions. I plan to explore ensembling both models to further reduce false negatives, dynamic threshold calibration to balance safety and usability, and extension to multiclass mushroom datasets. Semi-supervised techniques could leverage unlabeled mushroom data, and continuous online fine-tuning may adapt the models to regional species variations.

References